LAPORAN PERTEMUAN 4

Mengambil Data dari REST API

(HTTP, Async, dan FutureBuilder)



Dosen Pengampu:

JEFRY SUNUPURWA ASRI, S.Kom., M.Kom.

Disusun oleh:

Rizqi Bagas Wicaksono (20240801187)

Mata Kuliah:

Pemrograman Mobile (CR002)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

BAB I
PENDAHULUAN

1.1 Latar Belakang

Hampir seluruh aplikasi mobile modern tidak lagi bekerja dengan data yang tertanam di dalam aplikasi, melainkan mengambil data dari server melalui internet. Proses pengambilan data tersebut membutuhkan waktu dan bisa gagal karena berbagai sebab, misalnya koneksi terputus, server lambat, atau alamat yang salah. Karena itu, aplikasi harus mampu bekerja secara asinkron agar antarmuka pengguna (UI) tidak membeku, sekaligus menampilkan keadaan memuat (loading), keadaan galat (error), dan keadaan data berhasil dimuat dengan jelas kepada pengguna.

Pada Pertemuan 4 ini, praktikum difokuskan pada pengambilan data dari REST API menggunakan paket http, penguraian JSON ke dalam class model, serta penyajian data asinkron menggunakan widget FutureBuilder pada Flutter. API yang digunakan adalah JSONPlaceholder, yaitu API publik gratis yang disediakan untuk keperluan latihan.

1.2 Tujuan

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Menjelaskan pemrograman asinkron di Dart: Future, async, dan await.
2. Mengambil data dari REST API memakai paket http dan menguraikan JSON.
3. Membuat class model dengan konstruktor fromJson.
4. Menampilkan data asinkron dengan FutureBuilder lengkap dengan status loading, error, dan data.
5. Menangani kegagalan jaringan dan menyediakan tombol coba lagi.

BAB II
LANDASAN TEORI

2.1 Pemrograman Asinkron di Dart

Mengambil data dari internet membutuhkan waktu. Agar UI tidak membeku, Dart memakai **Future**, yaitu objek yang berisi hasil yang akan tersedia di kemudian waktu. Kata kunci **await** menunggu hasil sebuah Future tanpa memblokir UI, dan hanya boleh dipakai di dalam fungsi yang ditandai **async**.

2.2 REST API dan JSON

Aplikasi mengirim permintaan HTTP (misalnya GET) ke sebuah URL dan menerima respons berformat JSON. Kode status 200 berarti permintaan sukses, 404 berarti sumber tidak ditemukan, dan 500 berarti terjadi kesalahan pada server.

2.3 Konsep dan Fungsi yang Digunakan

Konsep	Fungsi
http.get(uri)	Mengirim permintaan GET dan mengembalikan Future<Response>
jsonDecode(body)	Mengubah teks JSON menjadi Map atau List Dart
fromJson	Konstruktor factory yang mengubah Map menjadi objek model
FutureBuilder	Widget yang membangun UI berdasarkan status sebuah Future
snapshot	Berisi status (connectionState), data (data), dan error (error)

Tabel 1. Konsep dan fungsi pada pengambilan data dari REST API

BAB III
HASIL PRAKTIKUM

3.1 Alat dan Bahan

- Flutter SDK, editor, dan perangkat Android (mode debug) dari pertemuan sebelumnya.
- Proyek baru yang dibuat dengan perintah flutter create praktikum_4.
- Koneksi internet dan API JSONPlaceholder (jsonplaceholder.typicode.com).

3.2 Bagian A: Mengenal Future dan FutureBuilder

Pada bagian ini dibuat fungsi ambilSalam() yang mensimulasikan pengambilan data selama 2 detik memakai Future.delayed. Hasilnya ditampilkan dengan FutureBuilder: selama Future belum selesai (ConnectionState.waiting) ditampilkan CircularProgressIndicator, jika terjadi galat ditampilkan pesan galat, dan jika selesai ditampilkan teks hasilnya. Objek Future dibuat di dalam initState(), bukan di dalam build(), agar tidak dibuat ulang setiap kali layar dibangun ulang.

```
Future<String> ambilSalam() async {
  await Future.delayed(const Duration(seconds: 2));
  return 'Halo dari masa depan!';
}

@override
void initState() {
  super.initState();
  _future = ambilSalam();   // dibuat sekali di initState()
}
```

3.3 Bagian B: Menyiapkan Paket HTTP

Paket http ditambahkan ke proyek dengan perintah berikut pada terminal di folder proyek.

```
flutter pub add http
```

Agar aplikasi versi release Android dapat mengakses internet, izin berikut ditambahkan pada berkas android/app/src/main/AndroidManifest.xml, tepat di atas tag <application.

```
<uses-permission android:name="android.permission.INTERNET"/>
```

3.4 Bagian C: Model dan Fungsi Pengambil Data

Data dari endpoint /users berupa daftar objek JSON dengan kolom id, name, email, phone,

dan website. Data tersebut dipetakan ke class Pengguna yang memiliki konstruktor factory fromJson. Fungsi ambilPengguna() mengirim permintaan GET dengan batas waktu 10 detik, melempar Exception jika kode status bukan 200, lalu menguraikan JSON menjadi List<Pengguna>. Exception tersebut nantinya ditangkap oleh FutureBuilder melalui snapshot.error.

```
factory Pengguna.fromJson(Map<String, dynamic> json) {
  return Pengguna(
    id: json['id'] as int,
    name: json['name'] as String,
    email: json['email'] as String,
    phone: json['phone'] as String,
    website: json['website'] as String,
  );
}

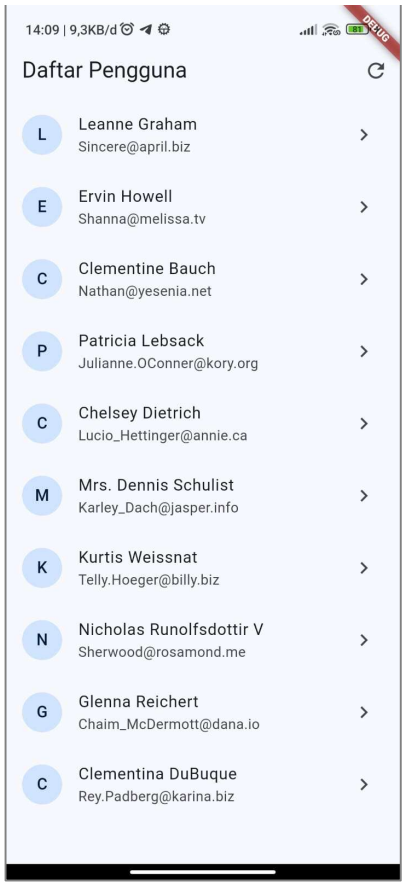
Future<List<Pengguna>> ambilPengguna() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/users');
  final response = await http.get(uri).timeout(const Duration(seconds: 10));
  if (response.statusCode != 200) {
    throw Exception('Gagal memuat data (kode ${response.statusCode})');
  }
  final List<dynamic> data = jsonDecode(response.body);
  return data.map((e) => Pengguna.fromJson(e as Map<String, dynamic>)).toList();
}
```

3.5 Bagian D: Menampilkan Data dengan FutureBuilder

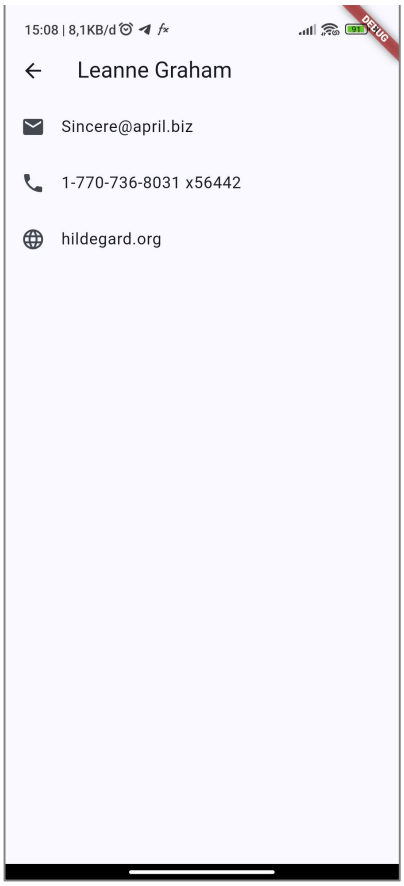
Halaman PenggunaPage memakai FutureBuilder yang menangani tiga keadaan: memuat (CircularProgressIndicator), galat (ikon, pesan, dan tombol Coba lagi), serta data (ListView.builder berisi ListTile). Tombol refresh pada AppBar memanggil _muatUlang(), yang membuat Future baru di dalam setState(). Hasilnya, 10 pengguna dari JSONPlaceholder tampil dalam daftar seperti pada Gambar 1. Setiap baris menampilkan inisial nama pada CircleAvatar, nama, email, dan ikon panah.

3.6 Bagian E: Halaman Detail

Ketika salah satu pengguna diketuk, Navigator.push membuka DetailPenggunaPage dengan membawa objek Pengguna yang dipilih. Halaman ini menampilkan email, nomor telepon, dan situs web pengguna dalam bentuk ListTile dengan ikon masing-masing, seperti pada Gambar 2 untuk pengguna Leanne Graham.



Gambar 1. Halaman Daftar Pengguna



Gambar 2. Halaman Detail Pengguna

3.7 Bagian F: Percobaan Penanganan Galat

Dua percobaan dilakukan untuk mengamati bagaimana aplikasi menangani kegagalan. Hasil yang diharapkan berdasarkan kode pada Bagian C dan D dirangkum pada Tabel 2.

No.	Percobaan	Tampilan yang muncul	Tindak lanjut
1	URL diubah menjadi /userz (halaman tidak ada)	Ikon galat merah dan teks "Terjadi kesalahan: Exception: Gagal memuat data (kode 404)" beserta tombol Coba lagi	URL dikembalikan ke /users, lalu tombol Coba lagi ditekan sehingga daftar pengguna tampil kembali
2	Koneksi internet perangkat dimatikan, lalu tombol refresh ditekan	Ikon galat merah dan pesan galat jaringan (berisi "Failed host lookup" / SocketException) beserta tombol Coba lagi	Internet dinyalakan kembali, lalu tombol Coba lagi ditekan sehingga data berhasil dimuat

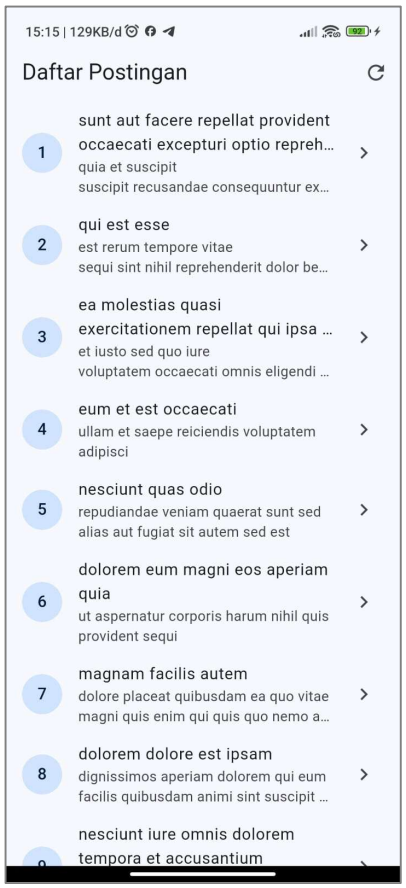
3.8 Tugas: Aplikasi Daftar Postingan

Tugas pertemuan ini adalah membuat aplikasi Daftar Postingan dengan endpoint /posts dan /posts/{id}/comments. Kode dipisah menjadi tiga bagian agar rapi: (1) model Post dan Komentar dengan fromJson, (2) layanan pengambil data (ApiService) yang tidak bergantung pada UI, dan (3) halaman UI yang memakai FutureBuilder. Rincian pemisahan tersebut ditunjukkan pada Tabel 3.

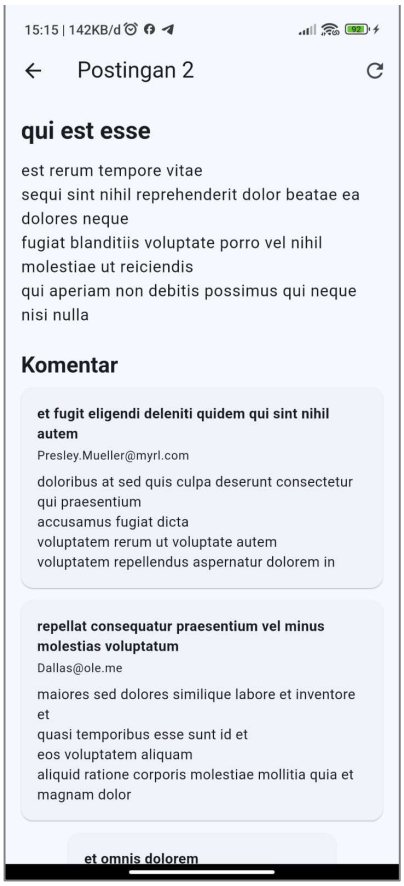
Bagian	Isi
Model	Class Post (userId, id, title, body) dan Komentar (postId, id, name, email, body), masing-masing dengan factory fromJson
Layanan	ApiService.ambilPost() mengambil /posts dan ApiService.ambilKomentar(postId) mengambil /posts/{id}/comments; keduanya memeriksa kode status dan melempar Exception bila bukan 200
UI	PostListPage (daftar postingan) dan PostDetailPage (isi lengkap dan komentar), serta widget ErrorView yang dipakai bersama untuk menampilkan galat dan tombol Coba lagi

Tabel 3. Pemisahan model, layanan, dan UI

Halaman utama (Gambar 3) menampilkan daftar postingan dengan nomor urut pada CircleAvatar, judul, dan potongan isi yang dibatasi dua baris dengan elipsis. Ketika sebuah postingan diketuk (contoh: postingan 2), halaman detail (Gambar 4) menampilkan judul dan isi lengkap postingan, kemudian bagian Komentar yang datanya diambil dari endpoint terpisah melalui Future kedua. Setiap komentar ditampilkan dalam sebuah kartu berisi nama, email, dan isi komentar. Kedua halaman memiliki tombol refresh pada AppBar, status loading, serta tampilan galat dengan tombol Coba lagi.



Gambar 3. Halaman Daftar Postingan



Gambar 4. Halaman Detail Postingan dan Komentar

BAB IV
PEMBAHASAN

Berikut jawaban atas pertanyaan refleksi pada modul praktikum.

4.1 Apa yang terjadi bila Future dibuat di dalam build() dan bukan di initState()?

Method build() dapat dipanggil berkali-kali, misalnya saat setState(), perubahan orientasi layar, atau perubahan tema. Jika Future dibuat di dalam build(), setiap pemanggilan akan membuat Future baru sehingga permintaan HTTP dikirim ulang, indikator loading muncul kembali, dan layar tampak berkedip. Hal ini boros kuota dan membebani server. Dengan membuat Future sekali di initState() dan menyimpannya dalam variabel, FutureBuilder cukup mengamati Future yang sama sehingga data hanya diambil satu kali sampai pengguna sengaja memuat ulang.

4.2 Apa perbedaan snapshot.hasError dengan memeriksa response.statusCode? Mengapa keduanya diperlukan?

response.statusCode diperiksa di dalam fungsi pengambil data untuk mengetahui apakah server menjawab dengan sukses. Server yang menjawab 404 atau 500 tetap dianggap "berhasil" oleh pustaka http karena respons diterima, sehingga tanpa pemeriksaan ini aplikasi akan mencoba menguraikan isi galat sebagai data. Sebaliknya, snapshot.hasError di sisi UI menandakan bahwa Future berakhir dengan galat, baik karena Exception yang dilempar kode kita (kode status bukan 200, JSON tidak sesuai) maupun karena kegagalan jaringan seperti SocketException atau TimeoutException. Keduanya diperlukan: pemeriksaan statusCode mengubah jawaban server yang gagal menjadi galat, sedangkan hasError memastikan semua jenis galat tersebut ditampilkan kepada pengguna.

4.3 Mengapa data JSON sebaiknya diubah menjadi class model, bukan dipakai langsung sebagai Map?

Class model memberi tipe data yang jelas sehingga kesalahan penulisan nama properti dapat terdeteksi saat kompilasi dan editor dapat memberikan autocomplete. Pada Map, kunci berupa string, sehingga salah ketik baru terlihat saat aplikasi berjalan dan nilai bertipe dynamic rawan memicu galat tipe. Selain itu, konversi dikumpulkan di satu tempat (fromJson), sehingga bila struktur JSON berubah, perbaikan cukup dilakukan pada model dan kode UI menjadi lebih

bersih serta mudah dirawat.

4.4 Mengapa UI wajib menyediakan status loading dan error, bukan hanya status data?

Pengambilan data dari jaringan tidak instan dan tidak selalu berhasil. Tanpa status loading, pengguna melihat layar kosong dan mengira aplikasi macet. Tanpa status error, aplikasi dapat menampilkan layar kosong atau bahkan crash ketika koneksi terputus, sehingga pengguna tidak tahu apa yang terjadi dan tidak memiliki cara untuk memperbaikinya. Dengan ketiga status (loading, error dengan tombol Coba lagi, dan data), aplikasi tetap informatif dan dapat dipulihkan oleh pengguna sendiri.

BAB V
KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Future, async, dan await memungkinkan pengambilan data dari internet dilakukan tanpa membekukan UI. Future sebaiknya dibuat di initState() agar tidak diulang setiap kali layar dibangun ulang.
2. Paket http bersama jsonDecode dapat dipakai untuk mengambil dan menguraikan data JSON dari REST API JSONPlaceholder, dan data tersebut dipetakan ke class model melalui konstruktor fromJson.
3. FutureBuilder memudahkan penyajian data asinkron karena satu widget dapat menangani tiga keadaan sekaligus: loading, error, dan data.
4. Kegagalan, baik karena kode status selain 200 maupun karena jaringan, dapat ditangani secara seragam melalui snapshot.error disertai tombol Coba lagi.
5. Aplikasi Daftar Postingan berhasil dibuat dengan model, layanan pengambil data, dan UI yang dipisahkan, serta dua Future terpisah untuk daftar postingan dan komentarnya.

LAMPIRAN

KODE PROGRAM TUGAS DAFTAR POSTINGAN

Lampiran 1. Model (Post dan Komentar)

```
class Post {
  final int userId;
  final int id;
  final String title;
  final String body;

  const Post({
    required this.userId,
    required this.id,
    required this.title,
    required this.body,
  });

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      userId: json['userId'] as int,
      id: json['id'] as int,
      title: json['title'] as String,
      body: json['body'] as String,
    );
  }
}

class Komentar {
  final int postId;
  final int id;
  final String name;
  final String email;
  final String body;

  const Komentar({
    required this.postId,
    required this.id,
    required this.name,
    required this.email,
    required this.body,
  });

  factory Komentar.fromJson(Map<String, dynamic> json) {
    return Komentar(
      postId: json['postId'] as int,
      id: json['id'] as int,
      name: json['name'] as String,
      email: json['email'] as String,
      body: json['body'] as String,
    );
  }
}
```

Lampiran 2. Layanan Pengambil Data (ApiService)

```
class ApiService {
  static const _base = 'https://jsonplaceholder.typicode.com';
```

```
  static Future<List<Post>> ambilPost() async {
    final response = await http
        .get(Uri.parse('$_base/posts'))
        .timeout(const Duration(seconds: 10));
    if (response.statusCode != 200) {
      throw Exception('Gagal memuat postingan (kode ${response.statusCode})');
    }
    final List<dynamic> data = jsonDecode(response.body);
    return data.map((e) => Post.fromJson(e as Map<String, dynamic>)).toList();
  }

  static Future<List<Komentar>> ambilKomentar(int postId) async {
    final response = await http
        .get(Uri.parse('$_base/posts/$postId/comments'))
        .timeout(const Duration(seconds: 10));
    if (response.statusCode != 200) {
      throw Exception('Gagal memuat komentar (kode ${response.statusCode})');
    }
    final List<dynamic> data = jsonDecode(response.body);
    return data
        .map((e) => Komentar.fromJson(e as Map<String, dynamic>))
        .toList();
  }
}
```

Lampiran 3. Widget Tampilan Galat

```
class ErrorView extends StatelessWidget {
  final Object error;
  final VoidCallback onRetry;
  const ErrorView({super.key, required this.error, required this.onRetry});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Icon(Icons.error_outline, size: 48, color: Colors.red),
            const SizedBox(height: 8),
            Text('Terjadi kesalahan:\n$error', textAlign: TextAlign.center),
            const SizedBox(height: 12),
            ElevatedButton(onPressed: onRetry, child: const Text('Coba lagi')),
          ],
        ),
      ),
    );
  }
}
```

Lampiran 4. Halaman Daftar Postingan

```
class PostListPage extends StatefulWidget {
  const PostListPage({super.key});
  @override
  State<PostListPage> createState() => _PostListPageState();
}

class _PostListPageState extends State<PostListPage> {
  late Future<List<Post>> _future;
```

```
  @override
  void initState() {
    super.initState();
    _future = ApiService.ambilPost();
  }

  void _muatUlang() => setState(() => _future = ApiService.ambilPost());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Daftar Postingan'),
        actions: [
          IconButton(icon: const Icon(Icons.refresh), onPressed: _muatUlang),
        ],
      ),
      body: FutureBuilder<List<Post>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return ErrorView(error: snapshot.error!, onRetry: _muatUlang);
          }
          final data = snapshot.data!;
          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, i) {
              final post = data[i];
              return ListTile(
                leading: CircleAvatar(child: Text('${post.id}')),
                title: Text(post.title, maxLines: 2,
                    overflow: TextOverflow.ellipsis),
                subtitle: Text(post.body, maxLines: 2,
                    overflow: TextOverflow.ellipsis),
                trailing: const Icon(Icons.chevron_right),
                onTap: () => Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (_) => PostDetailPage(post: post),
                  ),
                ),
              );
            },
          );
        },
      ),
    );
  }
}
```

Lampiran 5. Halaman Detail Postingan dan Komentar

```
class PostDetailPage extends StatefulWidget {
  final Post post;
  const PostDetailPage({super.key, required this.post});
  @override
  State<PostDetailPage> createState() => _PostDetailPageState();
}
```

```
class _PostDetailPageState extends State<PostDetailPage> {
  late Future<List<Komentar>> _future;

  @override
  void initState() {
    super.initState();
    _future = ApiService.ambilKomentar(widget.post.id);
  }

  void _muatUlang() => setState(
      () => _future = ApiService.ambilKomentar(widget.post.id));

  @override
  Widget build(BuildContext context) {
    final post = widget.post;
    return Scaffold(
      appBar: AppBar(
        title: Text('Postingan ${post.id}'),
        actions: [
          IconButton(icon: const Icon(Icons.refresh), onPressed: _muatUlang),
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text(post.title, style: Theme.of(context).textTheme.headlineSmall
              ?.copyWith(fontWeight: FontWeight.bold)),
          const SizedBox(height: 12),
          Text(post.body, style: Theme.of(context).textTheme.bodyLarge),
          const SizedBox(height: 24),
          Text('Komentar', style: Theme.of(context).textTheme.titleLarge
              ?.copyWith(fontWeight: FontWeight.bold)),
          const SizedBox(height: 8),
          FutureBuilder<List<Komentar>>(
            future: _future,
            builder: (context, snapshot) {
              if (snapshot.connectionState == ConnectionState.waiting) {
                return const Padding(
                  padding: EdgeInsets.all(24),
                  child: Center(child: CircularProgressIndicator()),
                );
              }
              if (snapshot.hasError) {
                return ErrorView(error: snapshot.error!, onRetry: _muatUlang);
              }
              return Column(
                children: snapshot.data!
                    .map((k) => Card(
                          child: Padding(
                            padding: const EdgeInsets.all(16),
                            child: Column(
                              crossAxisAlignment: CrossAxisAlignment.start,
                              children: [
                                Text(k.name, style: const TextStyle(
                                    fontWeight: FontWeight.bold)),
                                const SizedBox(height: 4),
                                Text(k.email),
                                const SizedBox(height: 8),
                                Text(k.body),
                              ],
                            ),
                          ),
                        ))
                    .toList(),
              );
            },
```

```
            ),
          ],
        ),
      );
  }
}
```